USD8 — History Compression and Observability Specification

Status: design specification, ready for implementation upon Cover Pool contract surface. **Source**: ports the on-chain primitive `vibeswap/contracts/libraries/IncrementalMerkleTree.sol` for the commitment layer; the bulk of the architecture below is the off-chain-storage + on-chain-commitment pattern, which is substrate-agnostic and well-established in modern zk-coprocessor designs. **Audience**: USD8 protocol team. Treat this as a porting brief plus a deliberate architectural commitment to off-chain linear scaling. The deepest design choice in the document — that bulk storage lives off-chain, on-chain holds only succinct commitments — is what makes the system scale to any holder count and any event count without an on-chain ceiling.

What this document is

USD8 needs to answer a specific scaling question. The Cover Score that determines a claim's payout is computed from the holder's USD8 usage history — "how much you've held and for how long." Today the history is implicit in the chain state, and the score is recomputed off-chain, signed by USD8's frontend, and verified on-chain at claim time. This works at the current holder count. It does not work at the holder count USD8 is designed to grow into, and it does not parallelize across multiple processors.

The naive on-chain pattern — store holder events in a mapping or in a fixed-depth Merkle tree — produces an on-chain ceiling at whatever depth or capacity is chosen at deployment. Beyond the ceiling, the protocol

either stops accepting events or requires a redeployment. Neither outcome is acceptable for a stablecoin designed to operate across decades.

The pattern that actually scales — and the pattern this document recommends — moves bulk storage off-chain and keeps the on-chain footprint to a succinct commitment per snapshot. Off-chain processors (indexers) ingest events from chain emissions and reconstruct full holder history in their own storage, parallelizing trivially across multiple instances. On-chain stores only the commitment root that summarizes all holder state at each snapshot. The architecture has no ceiling — it scales linearly with off-chain compute and storage capacity, and parallelizes by holder shard.

The good news is that the same shape of problem has been solved in our own codebase, that the on-chain commitment primitive (`IncrementalMerkleTree.sol`) is production-deployed, and that the off-chain layer follows the standard Brevis ProverNet pattern that USD8 has already chosen for Cover Score computation. The ingredients exist; the document below assembles them into the specific architecture USD8 should ship.

Section I — The scaling shape

USD8's per-holder usage history has a specific structure. Each holder has a sequence of balance-affecting events — mints, transfers in, transfers out, savings deposits, savings withdrawals — each with a timestamp and a delta. The Cover Score formula consumes this sequence and produces a scalar.

The shape that matters for compression analysis: - **Append-only**: events are added forward in time; never modified or removed retroactively. - **Per-actor partitioned**: each holder's events are independent of others' for the score computation. This is the property that enables off-chain parallelism — each indexer or each Brevis circuit can work on a single holder's history in isolation. - **Read-mostly at boundaries**: scores are queried at claim time, which is rare; events are written continuously,

which is the bulk of the load. - **Cryptographically verifiable**: any external party should be able to reproduce a claimant's score from the event log, with the on-chain contract verifying the reproduction. This is the Walkaway Test commitment applied to history.

The naive on-chain pattern — store events in a `mapping(address => Event[])` — has these costs: - **Storage**: O(events) per holder, indefinitely. Each event costs ~20,000 gas to write. - **Query**: O(events) per Cover Score recomputation. - **Ceiling**: bounded by gas economics; no path to scale past low-thousands of holders without prohibitive cost.

The fixed-depth on-chain Merkle pattern (which an earlier draft of this document recommended) is better but still ceiling-bounded — depth-20 caps at ~1M total events, and the depth choice is permanent at deployment. For a protocol that wants to operate without re-deployment over multi-decade horizons, this ceiling is the wrong shape.

The correct shape — and what the architecture below proposes — is *off-chain storage with on-chain commitment*. No ceiling. Linear scaling with off-chain capacity. Parallel by construction.

Section II — Survey of compression schemes

For completeness, this section reviews the candidate schemes for both layers — on-chain commitment and off-chain storage — and explains the choices in Section IV.

2.1 — On-chain commitment candidates

Sparse Merkle tree, indexed by holder address. Each holder occupies a 256-bit-keyed slot; slot value summarizes that holder's state (cumulative balance × time, last update timestamp, event count). Tree depth = 256 with sparse representation. On-chain stores root only; witness paths served off-chain. Witness size: O(log holders) ≈ 30 hashes

for any practical holder count. **This is the recommendation.** Production implementations exist (Quilibrium, Polygon zkEVM); audit-tractable.

Merkle Mountain Range (MMR), indexed by event sequence. Append-only structure supporting unbounded growth. New leaf appended in O(log n); witness for any historical leaf in O(log n). On-chain stores tip + recent roots. Used in Beam and various rollup designs. Solid alternative if USD8 prefers per-event commitments (each event is independently witnessable) over per-holder commitments.

Fixed-depth `IncrementalMerkleTree.sol` (the VibeSwap library). Append-only Merkle with fixed depth (typically 20 = 1M leaves) and Tornado-style 30-root ring buffer for async proof validity. Use case: if the snapshot scheme uses *batch commitments* — periodic transactions that commit a Merkle root of a snapshot's worth of state — the IncrementalMerkleTree can be the structure that holds the per-snapshot batch commitments themselves (depth 20 → 1M snapshots, more than 5 years at 1-minute snapshot cadence). This is the appropriate role for the VibeSwap primitive in the new architecture.

KZG polynomial commitments. Constant-size proofs (~48 bytes) but trusted setup, expensive pairing verification, and high implementation complexity. Not recommended for our claim cadence.

Verkle trees. Pre-production. Tooling immature. Not recommended yet.

RSA accumulators. Trusted setup, expensive verification, no membership/non-membership benefits we need. Not recommended.

2.2 — Off-chain storage candidates

Single-instance indexer (Subgraph / Ponder / custom). Simplest case: one indexer instance reads chain events, stores per-holder event timelines in a relational or key-value store. Sufficient for low-throughput protocols. Single point of failure.

Sharded multi-instance indexer. Multiple indexer instances each handle a subset of holders (shard by address-prefix). Each shard is independently reconstructable from chain events. No coordination required during reads — each shard is self-contained. **This is the recommendation for production.**

Decentralized indexing networks. Subsquid, The Graph (decentralized), or similar provide redundant indexing across multiple operators. Highest decentralization; appropriate when USD8 wants to remove the "trust the indexer" surface entirely.

The architecture below works with any of the three off-chain options. The choice between them is operational, not architectural.

Section III — Why this architecture works

The substrate-geometry argument is structural. USD8's history scaling problem has these properties:

- Append-only events with no retroactive modification → suits indexer reconstruction from chain log
- Per-holder partitioned computation → trivially parallelizable across processors
- Read-rare, write-continuous → write cost is the bottleneck, not read cost
- Walkaway Test required → events must be on-chain (as event logs) so anyone can reconstruct independently

The off-chain-storage + on-chain-commitment pattern matches all four properties: - Bulk storage off-chain → no on-chain ceiling - Indexers parallelize trivially by holder shard - Write cost on-chain is one event emission per balance change (≈10k gas) plus one commitment transaction per snapshot interval - Walkaway Test passes because every balance-affecting event is on-chain as an indexed event log; any party can spin up an indexer and reconstruct full history from chain alone

The pattern is not novel. It is the standard architecture for modern zk-coprocessor systems (Brevis, Axiom, Lagrange) and for many production rollups. We are recommending the well-trodden path because it has the substrate-match properties USD8 needs, not because it is interesting.

The earlier draft of this document recommended a fixed-depth on-chain tree for storage. That recommendation was wrong — it imposed an unnecessary on-chain ceiling on a problem whose correct solution is off-chain unbounded scaling. The corrected architecture below takes USD8 in the direction the protocol's actual constraints point.

Section IV — The proposed architecture

USD8's per-holder usage history is recorded as on-chain events emitted on every balance-affecting operation. Off-chain indexers (one or more, sharded by holder address) ingest events and reconstruct per-holder timelines in their own storage. At snapshot intervals (daily by default), an authorized snapshotter commits a sparse-Merkle-tree root summarizing all holder state at that moment. Brevis (or another zk-coprocessor) computes Cover Scores against the off-chain history and produces proofs that the on-chain contract verifies against the snapshot root.

4.1 — On-chain emission layer

Every USD8 balance-affecting operation emits a structured event:

```
event BalanceChange(
    address indexed holder,
    int128 balanceDelta,
    uint128 balanceAfter,
    uint8 eventType,   // 0=transfer, 1=mint, 2=burn, 3=savings deposit,
4=savings withdraw
    uint64 timestamp,
    bytes32 eventNonce
);
```

The event is emitted from the USD8 token contract's `_afterTokenTransfer` hook, so emission is automatic on every state change. No additional storage on-chain. Cost: one `LOG4` per transfer (~2,000 gas).

Replay protection is a per-event mapping if the protocol needs it for any externally-sourced events; for transfer-derived events, the ERC-20 transfer guarantees suffice.

4.2 — Off-chain storage layer

Indexers consume `BalanceChange` events and maintain per-holder timelines in their own storage. The storage shape is not part of the on-chain contract; it is operational infrastructure. The contract's only commitment is that every event is emitted, indexed, and queryable from chain log.

Sharding architecture: - Multiple indexer instances run in parallel - Each instance is assigned a shard of the holder address space (e.g., by first byte of address) - Each shard is reconstructable from chain log alone — no coordination between shards required - Per-holder query routes to the appropriate shard - Shard count can grow over time without re-indexing; new shards subscribe to the same chain log

This is linear scaling: doubling the holder count means doubling the shard count, with no per-shard increase in load. It is also fault-tolerant — a failed shard can be re-spun from chain log; a parallel redundant shard provides hot failover.

Aggregate metrics for fast off-chain queries (running cumulative balance × time integral, last-update timestamp, event count) are computed and cached per holder by the indexer. No on-chain caching is required because the on-chain contract does not serve aggregate queries — those are off-chain only.

4.3 — On-chain commitment layer

At snapshot intervals (daily recommended), the protocol commits a sparse Merkle tree root summarizing all holder state at that moment.

```
struct Snapshot {
    uint64 timestamp;
    uint128 totalSupply;
    bytes32 stateRoot;     // sparse Merkle root over holders
}

mapping(uint64 => Snapshot) public snapshots;  // keyed by snapshot
index
uint64 public lastSnapshotIndex;

IncrementalMerkleTree.Tree internal _snapshotRootTree;  // commits the
sequence of stateRoots

function commitSnapshot(
    uint64 snapshotIndex,
    uint64 timestamp,
    uint128 totalSupply,
    bytes32 stateRoot
) external onlySnapshotter {
    // Snapshotter computes stateRoot off-chain from indexer state
    // Posts the root on-chain in a single transaction
    snapshots[snapshotIndex] = Snapshot(timestamp, totalSupply,
stateRoot);
    lastSnapshotIndex = snapshotIndex;
    _snapshotRootTree.insertLeaf(stateRoot);  // commits to history of
roots
    emit SnapshotCommitted(snapshotIndex, timestamp, stateRoot);
}
```

The sparse Merkle tree itself lives off-chain; only its root reaches the chain. Witness paths for individual holders are served by indexers. The IncrementalMerkleTree of snapshot roots provides verifiable history of all snapshots ever committed — anyone can prove “snapshot N had root R” from the on-chain commitment chain.

Cost per snapshot: one transaction (~50k gas: one storage write + one IMT insert + one event emission). Daily cadence over five years: 1,800 snapshots, ~90M gas total ≈ negligible.

4.4 — Brevis integration

Brevis (or any zk-coprocessor) computes Cover Scores against the off-chain history. The proof binds to a specific snapshot:

```
function verifyAndAttestScore(
    address holder,
    uint256 claimedScore,
    uint64 snapshotIndex,
    bytes calldata brevisProof
) external returns (bool) {
    Snapshot memory snap = snapshots[snapshotIndex];
    if (snap.timestamp == 0) revert UnknownSnapshot();

    // brevisProof attests: "claimedScore is the correct Cover Score for
`holder`
    //                       computed from the holder's events as
committed in snap.stateRoot"
    if (!brevis.verifyScoreCircuit(holder, claimedScore, snap.stateRoot,
brevisProof)) {
        revert InvalidScoreProof();
    }

    emit ScoreAttested(holder, claimedScore, snapshotIndex);
    return true;
}
```

The Brevis circuit consumes (holder address, holder's event timeline from the indexer, the sparse-Merkle witness path proving the holder's state in the snapshot, the Cover Score formula). It produces a proof that the claimed score is correct against the committed root.

Parallelism: Brevis circuits can run concurrently for different holders, against different snapshots, on different prover instances. There is no serialization constraint. Multiple claims can be processed simultaneously without coordination.

4.5 — Why sparse Merkle by holder

The choice of sparse Merkle by holder (rather than by event sequence) matches the substrate. Cover Score queries are per-holder, not per-event. The natural witness shape is “prove this holder’s state in the snapshot” — one witness path of O(log holders) hashes. Per-event witnesses (which an MMR would provide) would require the verifier to re-compute the score from the events, increasing circuit complexity and proof size.

The MMR alternative is preserved as a fallback if USD8’s specific use case turns out to want per-event provenance. For Cover Score specifically, sparse Merkle by holder is the cleaner fit.

Section V — Walkaway Test compatibility

The architecture passes the Walkaway Test by construction. If USD8’s team disappears tomorrow:

- Every `BalanceChange` event is on-chain as an indexed event log. Any party can read them.
- Any party can spin up an indexer, replay the entire chain log, and reconstruct the full per-holder history. This is the same operation the original indexers performed; the chain log is the canonical source.
- The sparse Merkle tree construction is open-source (per the spec; not a USD8 trade secret). Any party can recompute the snapshot roots from their reconstructed history and verify them against the on-chain commitment chain.
- The Brevis circuit is open-source (per Brevis’s public-circuit policy). Any party can re-run it against any snapshot.
- The on-chain claim contract continues to function and verify proofs.

Anyone who wants to claim against the protocol can: 1. Spin up an indexer or query an existing one. 2. Generate the sparse-Merkle witness for their holder slot in the appropriate snapshot. 3. Run the Brevis circuit (or any

conforming implementation) to produce a score proof. 4. Submit the proof to the still-functioning claim contract. 5. Receive the payout.

No team intervention required at any step. The Walkaway Test passes because the chain log is the source of truth and every party has equal access to it.

This is a *stronger* Walkaway Test than the earlier on-chain-tree architecture, because there is no risk of an on-chain tree filling up and requiring team-led migration. The architecture has no ceiling that the team's continued operation defends against.

Section VI — Observability

The architecture creates a richer observability surface than naive storage, because the off-chain layer is the primary store rather than a derivative:

For dashboards / explorers: any indexer can serve aggregate queries (total supply by snapshot, holder count by tenure cohort, claim rates, Cover Score distribution) without on-chain calls. Indexer queries are O(1) for cached aggregates and O(log holders) for per-holder lookups.

For protocol analytics: the on-chain `Snapshot` struct exposes total supply per snapshot at O(1) read. Time-series queries are O(snapshot count) — for daily snapshots over five years, ~1,800 reads, fully cacheable in any analytics frontend.

For third-party auditors: the on-chain `_snapshotRootTree` provides verifiable history of all snapshot roots. Any auditor can request a Merkle proof of any historical snapshot root and verify it locally. Combined with an indexer's witness for any holder, the auditor can independently verify any Cover Score claim end-to-end.

For users: the USD8 frontend shows current Cover Score by querying any indexer (or running its own). Display does not require a Brevis proof — proofs are generated only at claim time.

The observability surface is wider than what naive storage provides because the indexer is a first-class component of the architecture, not an afterthought. Counterintuitively, moving storage off-chain *increases* what consumers can see cheaply, because the indexer is designed for queries while the chain is designed for verifiable commitments.

Section VII — What ports from VibeSwap

The following primitives are directly liftable from `vibeswap/contracts/`:

- **`IncrementalMerkleTree.sol` (199 LOC)**: zero modification needed for the *commitment-history* layer. The library commits the sequence of snapshot roots as an append-only Merkle tree, providing verifiable history of all snapshots ever committed. This is the appropriate role for the VibeSwap primitive in the new architecture — not bulk event storage, but commitment-of-commitments.
- **`ContributionAttestor.sol` state machine (508 LOC)**: porting target = `CoverScoreAttestor.sol`. The pending → attested → contested → escalated → resolved flow is generic and substrate-agnostic. Replaces "attest contribution" with "attest score"; the state machine is unchanged.
- **Replay protection patterns**: the `mapping(bytes32 => bool)` pattern for processed-event deduplication ports unchanged for any externally-sourced events USD8 needs to deduplicate.
- **Snapshotter authorization patterns**: the role-management surfaces in VibeSwap's CRA-like contracts port directly to the `commitSnapshot` authorization in the new architecture.

The following primitives do NOT port directly, but the reason in each case is more specific than "wrong substrate":

- **BFS trust traversal**: VibeSwap's trust score involves walking a vouch graph from a small founder set, with `MAX_TRUST_HOPS = 6` and a BFS queue capped at 1024 nodes. USD8 has no founder-rooted vouch

graph and no peer-attestation primitive — the *graph itself* doesn't exist in the USD8 substrate, so there's nothing to port. This is a substrate mismatch (USD8 doesn't have a graph), not a scaling gap.

- **On-chain per-event storage** (vouches, attestations, value-events): VibeSwap stores these on-chain inside bounded design envelopes — the trust graph is curated and capped; `MAX_ATTESTATIONS_PER_CLAIM = 50` ; the audit tree is depth-20 (1M leaves). These bounds are appropriate for VibeSwap's scale (hundreds-to-low-thousands of curated contributors with deliberate, low-frequency vouching). They are not appropriate for USD8 (potentially millions of holders, every transfer producing two balance-change events, easily tens of millions of events per year). The USD8 architecture moves storage off-chain not because the VibeSwap *pattern* is wrong, but because the VibeSwap *scale envelope* doesn't fit USD8's expected throughput.

The patterns themselves — the attestation state machine, the Merkle commitment primitive, the rate-limited recalculation discipline, the replay-protection mapping — all port directly. The change is the storage location of the bulk per-event data, not the design of the surrounding mechanisms.

Worth flagging for VibeSwap's own future: the same scaling pressure that forces USD8 off-chain will eventually arrive in VibeSwap if adoption follows the Cognitive Economy Thesis to full propagation. The current on-chain storage envelopes are explicit markers of where the ceiling lives. The off-chain-storage + on-chain-commitment refactor specified in this document is a template that ports *back* to VibeSwap whenever that pressure arrives. Banked as a future VibeSwap workstream; not blocking anything today.

The summary: the `IncrementalMerkleTree` library, the attestor state machine, and the supporting patterns port directly. The bulk of the work is the off-chain indexer infrastructure and the Brevis circuit, which are USD8-specific and not ports. The storage-location pivot — moving per-event data off-chain — is the architectural choice forced by USD8's scale envelope, not a critique of VibeSwap's existing design.

Section VIII — Implementation phases

Phase 1 — On-chain emission layer (1 day) - Wire `BalanceChange` event into USD8 token's `_afterTokenTransfer` hook. - Tests: event emission on every balance-affecting operation; gas overhead measurement. - No new contracts; minimal change.

Phase 2 — Off-chain indexer (1–2 weeks) - Build sharded indexer (e.g., Ponder-based with address-prefix sharding). - Per-shard storage: holder timeline + cached aggregates. - API for per-holder queries + sparse-Merkle witness generation. - Operational: deploy multiple indexer instances for redundancy.

Phase 3 — Snapshot commitment contract (3–5 days) - Write `USD8SnapshotCommitter.sol` with `commitSnapshot` + read functions. - Wraps `IncrementalMerkleTree.sol` for the snapshot-root history. - Tests: snapshot commitment, root query, IMT integration, gas-cost measurement.

Phase 4 — Snapshotter off-chain process (1 week) - Off-chain process that periodically reads indexer state, computes the sparse Merkle root, posts on-chain. - Configurable cadence (daily default). - Permissionless or keeper-based per USD8 preference.

Phase 5 — Brevis circuit for Cover Score (1–2 weeks) - Spec the circuit: inputs (holder address, event timeline, witness path, formula constants); output (score). - Implement against Brevis SDK. - Test against synthetic histories of varying sizes.

Phase 6 — On-chain verifier integration (3–5 days) - Write `CoverScoreAttestor.sol` (port of VibeSwap's `ContributionAttestor.sol` with Brevis-verification wired in). - Connect to claim flow.

Phase 7 — Migration of pre-existing holder history (1 week) - For holders existing pre-deployment, the indexer reads the historical chain log (transfer events from token deployment forward) and reconstructs

timelines. - No special migration code needed — the indexer handles historical events the same as new ones. - This is much simpler than the earlier on-chain migration pattern.

Total estimated wall-clock: 6–8 weeks for a small implementation team. The Brevis circuit (Phase 5) is the most novel work; the rest is well-established patterns.

Section IX — Open questions for the USD8 team

1. **Snapshot cadence**. Daily is the recommendation. Hourly is also tractable but produces ~24× more on-chain commitment transactions. Weekly is cheaper but loses lookback granularity. Consider whether claim-frequency requires finer than daily.

2. **Sparse Merkle library choice**. Multiple production-grade implementations exist (Quilibrium, Polygon zkEVM, OpenZeppelin's emerging library). Choice affects audit budget and Brevis circuit compatibility.

3. **Indexer architecture**. Recommendation is sharded multi-instance (Section 4.2). Single-instance is simpler for launch; decentralized indexing network (Subsquid / decentralized Graph) is most resilient. Choose based on operational tolerance for off-chain trust.

4. **Snapshotter authorization model**. Single trusted snapshotter (simplest)? Multi-sig? Permissionless with bond-based quality assurance? The snapshotter cannot lie about what's on-chain (the indexer can verify) but can choose timing. Recommendation: keeper-based, permissionless with cooldown.

5. **Cover Score formula stability**. Hard-coded constants in the Brevis circuit, or governance-tunable parameters? Tunable parameters require re-circuit-compilation on each governance change; hard-coded constants require constitutional-amendment process to change.

6. **Brevis vs. alternative zk-coprocessors**. Brevis is the natural choice given USD8's existing partnership. RISC Zero, Succinct's SP1, Axiom, Lagrange are alternatives. The architecture is coprocessor-agnostic — any prover that produces verifiable computation against a Merkle-rooted state will work.

7. **Indexer operator economics**. For sharded multi-instance, who runs the shards? USD8 team? Independent operators paid in USD8? Decentralized indexing network? Decentralization tradeoff.

8. **Historical migration depth**. How far back should the indexer reconstruct? Token-genesis to present is the default; if USD8 wants to limit history depth (for privacy or computational reasons), the snapshot can declare a "history cutoff" beyond which events do not contribute to scores.

Appendix A — Why the time-bucketed aggregate works

The Cover Score formula in USD8's existing copy says scores are based on "how much you've held and for how long." The natural mathematical form for this is an integral of held balance over time:

\[S(, T) = _0^T b(, t) d(t), dt \]

where b(holder, t) is the holder's balance at time t and $d(t)$ is a (possibly trivial) discount factor.

For an event-log representation, balance is piecewise constant between events. Between event i at t_i and event $i + 1$ at $t_{i + 1}$, balance is b_i. The integral becomes a sum:

\[S(, T) = *i b_i (t*{i+1} - t_i) d(t_i) \]

The off-chain aggregate maintained by the indexer — `totalUsdHeld` — is exactly this sum (without the discount factor; the discount is applied off-chain by Brevis at claim time). Each new event updates the aggregate in

O(1) per indexer. The Brevis circuit consumes the aggregate plus a small number of historical events (those needed for discount-factor application) and produces the discounted score.

The architecture amortizes the cost of summation across the writes (cheap per-event indexer update) rather than concentrating it at read time (expensive per-claim computation). This is the right tradeoff because writes are continuous and reads are rare.

Appendix B — What we considered, with revised reasons

For the audit-conversation that will eventually happen, here is the corrected analysis of candidate architectures:

Scheme	Verdict	Reason
Off-chain history + on-chain commitment	**RECOMMENDED**	Linear scaling; parallelizable; no on-chain ceiling; Walkaway Test passes via on-chain event emissions.
Sparse Merkle tree (on-chain commitment layer)	RECOMMENDED for snapshot commitments	O(log holders) witness; substrate-matched to per-holder Cover Score queries.
`IncrementalMerkleTree.sol` for snapshot-root history	RECOMMENDED	Inherits VibeSwap's audit history; appropriate role is committing the *sequence of*

Scheme	Verdict	Reason
		snapshot roots, not raw events.
Merkle Mountain Range (event-indexed)	Acceptable alternative	Better suited if per-event provenance is needed; sparse Merkle by holder is cleaner for the Cover Score query shape.
Fixed-depth on-chain tree storing raw events	REJECTED	Imposes an unnecessary on-chain ceiling. The earlier draft of this document recommended this; the recommendation was wrong.
Plain `mapping(address => Event[])`	REJECTED	O(events) state and O(events) query cost on-chain; doesn't scale past low-thousands of holders.
KZG polynomial commitments	REJECTED	Trusted setup; high pairing cost; complexity exceeds benefit.
Verkle trees	REJECTED	Pre-production; tooling immature.

Scheme	Verdict	Reason
RSA accumulators	REJECTED	Trusted setup; expensive verification; non-membership not needed.

The earlier draft of this document recommended a fixed-depth on-chain tree with the off-chain pattern explicitly rejected (citing a Walkaway Test concern that was misplaced — the chain event log is the reconstructible source, not the on-chain tree). The recommendation has been corrected here. The off-chain pattern was rejected for the wrong reason; the corrected analysis shows it is the right answer.

Specification authored by William Glynn with primitive-assist from JARVIS. Source on-chain primitive: `vibeswap/contracts/libraries/IncrementalMerkleTree.sol` *(production, deployed). The architecture is the standard off-chain-storage + on-chain-commitment pattern used in modern zk-coprocessor systems, applied to USD8's specific Cover Score query shape with sparse-Merkle-by-holder for the commitment layer. Open to refinement on any specific phase as Rick's team determines what fits their architecture and audit budget. Implementation will commence upon access to USD8 contract surface and confirmation of the open questions in Section IX.*